

IMPLEMENTÁCIA PREKLADAČA JAZYKA IFJ25

Tým xmezeim00, Varianta BVS (Binárny vyhľadávací strom)

Členovia tímu

Meno	Login	Rozdelenie bodov (Taktiež v súbore rozdeleni)
Martin Mezei (vedúci)	xmezeim00	26%
Martin Raček	xracekm00	26%
Kristián Cilling	xcillik00	22%
Samuel Fačka	xfackas00	26%

Implementované rozšírenia

FUNEXP | CYCLES

Práca v tíme

Každý štvrtok večer sme mali tímový meeting v priestoroch školy, kde sme mali rezervovanú miestnosť. Spoločne sme navrhovali jednotlivé časti kompilátora, rozdeľovali si úlohy a riešili možnosti implementácie. Mimo osobných stretnutí sme komunikovali cez Discord a používali Git na správu verzií.

Rozdelenie práce

Pracovali sme tímovo a každý sa podieľal na viacerých úlohách – od návrhu, cez implementáciu až po testovanie a debugovanie. Primárne rozdelenie však vyzeralo nasledovne:

Martin Mezei (xmezeim00):

- Návrh konečného automatu pre scanner
- Implementácia tabuľky symbolov (BVS)
- Implementácia sémantickej analýzy
- Návrh LL gramatiky
- Spolupráca na generovaní kódu
- Testovanie a oprava chýb

Martin Raček (xracekm00):

- Implementácia lexikálnej analýzy (scanner.c)
- Návrh konečného automatu
- Tvorba Makefile
- Implementácia generovania kódu
- Spolupráca na návrhu generovania kódu
- Ladenie a optimalizácia scanner
- Návrh abstraktného syntaktického stromu
- Testovanie a oprava chýb

Samuel Fačka (xfackas00):

- Implementácia syntaktickej analýzy (parser.c)
- Návrh LL gramatiky
- Implementácia abstraktného syntaktického stromu (AST)
- Rekurzívny zostup pre príkazy
- Testovanie parsera , generovania kódu a abstraktného syntaktického stromu
- Rozšírenie precedenčnej analýzy, oprava chýb

Kristián Cilling (xcillik00):

- Návrh a prepísanie FSM do Draw.io
- Implementácia precedenčného analyzátora (PSA)
- Implementácia generovania kódu pre build in funkcie
- Tvorba dokumentácie
- Návrh precedenčnej tabuľky
- Testovanie výrazov a generovanie kódu

Lexikálna analýza

Návrh a Implementácia

Ako prvé sme začali s lexikálnym analyzátorom. Najskôr sme si navrhli niekoľko FSM (konečných automatov), ktoré reprezentovali rôzne typy tokenov. Na základe týchto FSM sme implementovali jednotlivé funkcie parsujúce vstupný program na tokeny.

Vytvorili sme si dátovú štruktúru `token_t`, ktorá obsahuje:

- `type` – typ tokenu (enum `token_type`)
- `value` – union obsahujúci hodnotu (`int_value`, `float_value`, `str_value`, `other_value`)
- `ast` – ukazovateľ na AST uzol (pre integráciu s PSA)

Scanner používa globálne premenné `has_been_pushed` a `pushed_token` na sledovanie stavu pre podporu lookahead funkcionality, ktorá je potrebná pri syntaktickej analýze. Taktiež implementuje globálne premenné `eof_reached` a `pending_eof_token` pre správne spracovanie konca súboru pri lookahead operáciách.

Pre prácu so scannerom sú implementované štyri hlavné funkcie:

- `get_token()` – načíta a dekoduje ďalší token zo vstupného kódu, vracia pointer na novo alokovanú štruktúru tokenu
- `push_token()` – vráti token späť do scanneru, používa sa pri lookahead v parseri
- `free_token()` – uvoľní pamäť alokovanú pre token, vrátane dynamicky alokovaných reťazcov
- `scanner_cleanup()` – uvoľní všetky potrebné alokované zdroje po ukončení práce scanneru

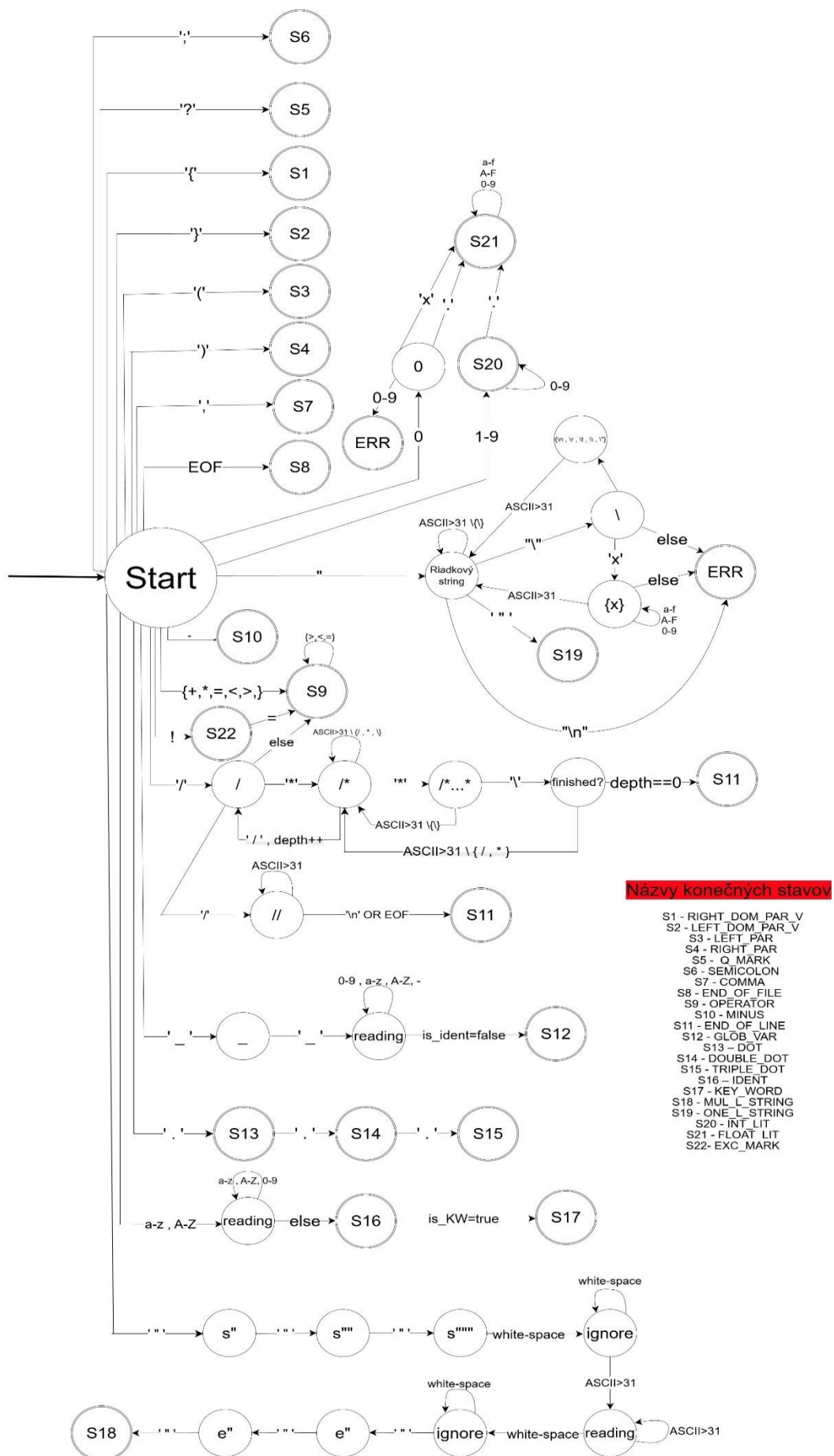
V module `lex_funs.c` sú implementované špecializované funkcie pre spracovanie rôznych typov tokenov napr.:

- `process_next_token()` – hlavná funkcia rozhodujúca o type tokenu na základe prvého znaku
- `process_ident()` – spracováva identifikátory a kľúčové slová

Scanner používa globálne pole kľúčových slov `key_words_arr[]` pre identifikáciu kľúčových slov jazyka. Pri spracovaní identifikátorov sa kontroluje, či sa nejedná o kľúčové slovo.

Celý lexikálny analyzátor je implementovaný v súboroch `scanner.c/.h` (hlavné funkcie pre používateľa scanneru) a `lex_funs.c/.h` (pomocné funkcie pre spracovanie konkrétnych typov tokenov).

FSM pre scanner



Syntaktická analýza

Návrh a Implementácia

Syntaktická analýza je rozdelená na dva moduly – LL syntaktická analýza pre príkazy a štruktúru programu a precedenčná syntaktická analýza pre spracovanie výrazov.

Najskôr sme si navrhli prototyp LL gramatiky do diagramov, z ktorých sme neskôr vytvorili kostru parseru. Potom sme gramatiku upravili, aby spĺňala pravidlá LL gramatiky. Následne podľa pravidiel boli implementované funkcie parsujúce jednotlivé gramatické konštrukcie.

Parser používa metódu rekurzívneho zostupu. Hlavná funkcia `parse_program()` rekurzívne spúšťa všetky potrebné funkcie na základe prijatých tokenov. Parser využíva moduly `syntable.c/.h` na prácu s tabuľkou symbolov, kam ukladá definície funkcií.

Implementované sú pomocné funkcie pre prácu s tokenmi:

- `expect_type()` – očakáva token daného typu, pri nesúhlase volá `error_exit()`
- `expect_ident()` – očakáva identifikátor
- `look_ahead()` – implementuje lookahead funkcionality – načíta ďalší token a vráti ho späť do scanneru
- `consume_eols()` – preskočí prázdne riadky (EOL tokeny), používa sa po operátoroch a zátvorkách

Parser buduje abstraktný syntaktický strom (AST) počas syntaktickej analýzy. Pre každú gramatickú konštrukciu (definícia funkcie, blok príkazov, príkaz atď.) sa vytvára príslušný uzol AST.

Hlavné funkcie parseru zahŕňajú:

- `parse_program()` – vstupný bod parsera, parsuje celý program
- `parse_prolog()` – spracuje prolog (`import "ifj25" for Ifj`)
- `parse_class_def()` – spracuje definíciu triedy `Program`
- `parse_definition()` – rozlíši typ definície (funkcia/getter/setter)
- `parse_block()` – spracuje blok príkazov
- `parse_statement()` – spracuje jednotlivé príkazy (`if`, `while`, `for`, `return`, `break`, `continue`, definícia premennej, priradenie)

Pri spracovávaní výrazov parser volá funkciu `parse_expression()` z modulu precedenčnej analýzy. Táto funkcia prijíma parameter `recognition_token`, ktorý špecifikuje kontext výrazu (napr. pravá strana priradenia, výraz v `return`, výraz v `for` cykle).

Syntaktický analyzátor je implementovaný v súboroch `parser.c/.h` (hlavný parser s rekurzívnym zostupom) a `parser_expression.c/.h` (precedenčný analyzátor pre výrazy).

LL Gramatika

Pravidlá gramatiky:

1. $\langle \text{Program} \rangle \rightarrow \langle \text{Prolog} \rangle \langle \text{ClassDef} \rangle \text{EOF}$
2. $\langle \text{Prolog} \rangle \rightarrow \text{"import" STRING_LITERAL "for" ID EOL}$
3. $\langle \text{ClassDef} \rangle \rightarrow \text{"class" "Program" "{" EOL} \langle \text{ClassBody} \rangle \text{"}" } \langle \text{Eols} \rangle$
4. $\langle \text{Eols} \rangle \rightarrow \text{EOL } \langle \text{Eols} \rangle$
5. $\langle \text{Eols} \rangle \rightarrow \varepsilon$
6. $\langle \text{ClassBody} \rangle \rightarrow \langle \text{StaticDefs} \rangle$
7. $\langle \text{StaticDefs} \rangle \rightarrow \text{"static" } \langle \text{Definition} \rangle \langle \text{StaticDefs} \rangle$
8. $\langle \text{StaticDefs} \rangle \rightarrow \varepsilon$
9. $\langle \text{Definition} \rangle \rightarrow \text{ID } \langle \text{DefTail} \rangle$
10. $\langle \text{DefTail} \rangle \rightarrow \text{"(" } \langle \text{ParamList} \rangle \text{"}" } \langle \text{Block} \rangle$
11. $\langle \text{DefTail} \rangle \rightarrow \text{"=" } \text{"(" ID "}" } \langle \text{Block} \rangle$
12. $\langle \text{DefTail} \rangle \rightarrow \langle \text{Block} \rangle$
13. $\langle \text{ParamList} \rangle \rightarrow \varepsilon$
14. $\langle \text{ParamList} \rangle \rightarrow \text{ID } \langle \text{ParamListTail} \rangle$
15. $\langle \text{ParamListTail} \rangle \rightarrow \text{"," ID } \langle \text{ParamListTail} \rangle$
16. $\langle \text{ParamListTail} \rangle \rightarrow \varepsilon$
17. $\langle \text{Block} \rangle \rightarrow \text{"{" EOL } \langle \text{StatementList} \rangle \text{"}" }$
18. $\langle \text{StatementList} \rangle \rightarrow \langle \text{StmtListTail} \rangle$
19. $\langle \text{StmtListTail} \rangle \rightarrow \langle \text{Block} \rangle \langle \text{StmtListTail} \rangle$
20. $\langle \text{StmtListTail} \rangle \rightarrow \langle \text{Statement} \rangle \langle \text{StmtListTail} \rangle$
21. $\langle \text{StmtListTail} \rangle \rightarrow \varepsilon$
22. $\langle \text{Statement} \rangle \rightarrow \langle \text{VarDef} \rangle \text{EOL}$
23. $\langle \text{Statement} \rangle \rightarrow \langle \text{AssignmentOrCall} \rangle \text{EOL}$
24. $\langle \text{Statement} \rangle \rightarrow \langle \text{IfStmt} \rangle \text{EOL}$
25. $\langle \text{Statement} \rangle \rightarrow \langle \text{WhileStmt} \rangle \text{EOL}$
26. $\langle \text{Statement} \rangle \rightarrow \langle \text{ReturnStmt} \rangle \text{EOL}$
27. $\langle \text{Statement} \rangle \rightarrow \langle \text{ForStmt} \rangle \text{EOL}$
28. $\langle \text{Statement} \rangle \rightarrow \langle \text{BreakStmt} \rangle \text{EOL}$
29. $\langle \text{Statement} \rangle \rightarrow \langle \text{ContinueStmt} \rangle \text{EOL}$
30. $\langle \text{Statement} \rangle \rightarrow \text{EOL}$
31. $\langle \text{VarDef} \rangle \rightarrow \text{"var" ID}$
32. $\langle \text{AssignTarget} \rangle \rightarrow \text{ID}$
33. $\langle \text{AssignTarget} \rangle \rightarrow \text{GLOBAL_ID}$
34. $\langle \text{AssignmentOrCall} \rangle \rightarrow \langle \text{AssignTarget} \rangle \text{"=" } \langle \text{Rhs} \rangle$
35. $\langle \text{IfStmt} \rangle \rightarrow \text{"if" "(" } \langle \text{Eols} \rangle \langle \text{Expression} \rangle \text{"}" } \langle \text{Block} \rangle \text{"else" } \langle \text{Block} \rangle$
36. $\langle \text{WhileStmt} \rangle \rightarrow \text{"while" "(" } \langle \text{Eols} \rangle \langle \text{Expression} \rangle \text{"}" } \langle \text{Block} \rangle$
37. $\langle \text{ReturnStmt} \rangle \rightarrow \text{"return"}$
38. $\langle \text{ReturnStmt} \rangle \rightarrow \text{"return" } \langle \text{Expression} \rangle$
39. $\langle \text{ForStmt} \rangle \rightarrow \text{"for" "(" } \langle \text{Eols} \rangle \text{ID "in" } \langle \text{Expression} \rangle \text{"}" } \langle \text{Block} \rangle$
40. $\langle \text{BreakStmt} \rangle \rightarrow \text{"break"}$
41. $\langle \text{ContinueStmt} \rangle \rightarrow \text{"continue"}$
42. $\langle \text{Rhs} \rangle \rightarrow \langle \text{Expression} \rangle$

LL Tabulka

	"import"	"class"	"static"	"var"	ID	GLOBAL_ID	"if"	"while"	"return"	"for"	"break"	"continue"	"{"	"}"	"("	")"	","	"="	"in"	EOL	EOF
<Program>	1																				
<Prolog>	2																				
<ClassDef>		3																			
<Eols>					5															4	5
<ClassBody>			6											6							
<StaticDefs>			7											8							
<Definition>					9																
<DefTail>													12		10			11			
<ParamList>					14											13					
<ParamListTail>																16	15				
<Block>													17								
<StatementList>				18	18	18	18	18	18	18	18	18	18	18						18	
<StmListTail>				20	20	20	20	20	20	20	20	20	19	21						20	
<Statement>				22	23	23	24	25	26	27	28	29								30	
<VarDef>				31																	
<AssignTarget>					32	33															
<AssignmentOrCall>					34	34															
<IfStmnt>							35														
<WhileStmnt>								36													
<ReturnStmnt>									37												
<ForStmnt>										39											
<BreakStmnt>											40										
<ContinueStmnt>												41									
<Rhs>					42	42									42						

Precedenčná analýza

Návrh a Implementácia

Pre spracovanie výrazov používame precedenčnú syntaktickú analýzu zdola-nahor. PSA spracováva aritmetické operátory (+, -, *, /), relačné operátory (<, >, <=, >=, ==, !=), typový operátor (is), rozsahové operátory (.., ...) a volania funkcií.

Precedenčná tabuľka definuje vzťahy medzi operátormi pomocou enumu `precedence_relation`. Pre indexovanie do precedenčnej tabuľky používame enum `precedence_index`, ktorý mapuje tokeny scanneru na indexy v tabuľke. Práca s PSA tabuľkou funguje pomocou týchto dvoch enumov.

PSA používa vlastnú implementáciu zásobníka, ktorý je realizovaný ako jednosmerný zretazený zoznam (singly linked list). Štruktúra Stack uchováva ukazovatele na:

- head – začiatok zoznamu (dno zásobníka)
- top – koniec zoznamu (vrchol zásobníka)
- top_terminal – najvrchnejší terminál v zásobníku

Zásobník mimo klasických funkcií taktiež podporuje:

- stack_push() – vloží token na vrchol zásobníka (na koniec zoznamu)

- `stack_pop()` – odstráni token z vrcholu zásobníka (z konca zoznamu)
- `stack_push_after()` – špeciálna funkcia, ktorá vkladá marker '<' za najvrchnejší terminál
- `stack_set_top_terminal_pointer()` – nastaví ukazovateľ na najvrchnejší terminál
- `stack_pop_no_free()` – odstráni položku bez uvoľnenia tokenu (pre reduce operáciu)

PSA používa špeciálne interné tokeny:

- **MARKER** – reprezentuje '<' marker vložený na zásobník
- **NONTERMINAL_E** – reprezentuje neterminál E, v poli ast tokenu uchováva ukazovateľ na koreň AST podstromu

Hlavná funkcia `parse_expression()` prijíma parameter `recognition_token`, ktorý špecifikuje kontext, v ktorom sa výraz parsuje. Pomocou `recognition_tokenu` sa rozhoduje kedy končí výraz. Funkcia inicializuje zásobník so znakom '\$', opakovane používa precedenčnú tabuľku pre shift/reduce operácie a na konci vracia koreň AST reprezentujúci spracovaný výraz.

Precedenčný analyzátor je implementovaný v súboroch `parser_expression.c/.h` a zásobník v súboroch `stack.c/.h`.

Precedenčná tabuľka definuje vzťahy medzi operátormi:

- `<` (shift) – ak má operátor na zásobníku nižšiu prioritu
- `>` (reduce) – ak má operátor na zásobníku vyššiu prioritu
- `=` (`eq_reduce`) – pre párové operátory ako zátvorky
- **Err** – chybná kombinácia
- **finish** – znamená úspešné ukončenie precedencie

ZJEDNODUSENA PRECEDENCNA TABULKA:

	<code>+-</code>	<code>*/</code>	<code><;>;<=>=</code>	<code>== ; !=</code>	<code>is</code>	<code>.. ...</code>	<code>(</code>	<code>)</code>	<code>i</code>	<code>\$</code>
<code>+-</code>	<code>></code>	<code><</code>	<code>></code>	<code><</code>	<code><</code>	<code><</code>	<code><</code>	<code>></code>	<code><</code>	<code>></code>
<code>*/</code>	<code>></code>	<code>></code>	<code>></code>	<code>></code>	<code><</code>	<code><</code>	<code><</code>	<code>></code>	<code><</code>	<code>></code>
<code><><=>=</code>	<code><</code>	<code><</code>	Err	<code>></code>	<code><</code>	<code><</code>	<code><</code>	<code>></code>	<code><</code>	<code>></code>
<code>==!=</code>	<code><</code>	<code><</code>	<code><</code>	Err	<code><</code>	<code><</code>	<code><</code>	<code>></code>	<code><</code>	<code>></code>
<code>is</code>	<code>></code>	<code>></code>	<code>></code>	<code>></code>	Err	<code><</code>	<code><</code>	<code>></code>	<code><</code>	<code>></code>
<code>.. ...</code>	<code>></code>	<code>></code>	<code>></code>	<code>></code>	<code>></code>	<code><</code>	<code><</code>	<code>></code>	<code><</code>	<code>></code>
<code>(</code>	<code><</code>	<code><</code>	<code><</code>	<code><</code>	<code><</code>	<code><</code>	<code><</code>	<code>=</code>	<code><</code>	Err
<code>)</code>	<code>></code>	<code>></code>	<code>></code>	<code>></code>	<code>></code>	<code>></code>	<code>></code>	Err	Err	<code>></code>
<code>i</code>	<code>></code>	<code>></code>	<code>></code>	<code>></code>	<code>></code>	<code>></code>	<code>></code>	Err	Err	<code>></code>
<code>\$</code>	<code><</code>	<code><</code>	<code><</code>	<code><</code>	<code><</code>	<code><</code>	<code><</code>	Err	<code><</code>	finish

Abstraktný syntaktický strom

Návrh a Implementácia

AST je navrhnutý ako univerzálna stromová dátová štruktúra. Všetky elementy sú reprezentované jedným typom `ASTNode`, ktorý uchováva informácie:

- `type` – typ uzla určujúci gramatickú konštrukciu
- `children[]` – pole ukazovateľov na detské uzly
- `child_count` – počet detských uzlov
- `union data` – union obsahujúci špecifické dáta pre konkrétne typy uzlov

Hlavná funkcia `ast_create()` vytvára nový uzol na základe parametra `type`, ktorý sa volí podľa práve spracovanej gramatiky v syntaktickej analýze. Funkcia využíva pomocnú funkciu `add_child()`, ktorá slúži na ukladanie uzlov do poľa detí.

Podstromy obsahujúce výrazy spracované v precedenčnej analýze sa vytvárajú ako binárne nevyvážené stromy. Pre konzistentnosť implementácie sú deti týchto podstromov taktiež ukladané do poľa detských uzlov. Implementovaná je aj funkcia `ast_free()`, ktorá rekurzívne uvoľní celú štruktúru AST.

Abstraktný syntaktický strom je implementovaný v súboroch `ast.c/.h`.

Sémantická analýza

Návrh a Implementácia

Sémantický analyzátor pracuje so štruktúrou AST a na základe typu aktuálne spracovávaného uzlu vykoná danú sémantickú kontrolu. Prechod abstraktným stromom je implementovaný ako PREorder prechod. Do sémantických kontrol spadajú veci ako napríklad:

- Overenie existencie funkcií a premenných
- Overenie redefinícií premenných a funkcií
- Kontrola správnosti výrazov a jednoduchá statická typová kontrola
- Kontrola argumentov pri volaní vstavaných funkcií `lfj`

Tabuľka symbolov a scope stack

Návrh a Implementácia

Pre implementáciu tabuľky symbolov sme si zvolili variant BVS. Implementovali sme ju ako AVL strom. Každý uzol ukladá informácie o symbole v ňom uloženom a to:

- meno premennej
- počet parametrov funkcie
- typ symbolu

Uzly tabuľky symbolov tiež uchovávajú informácie dôležité pre vyvažovanie stromu a ukazovatele na detské uzly. Knižnica pre prácu s tabuľkou symbolov implementuje všetky základné funkcie okrem odstraňovania uzlov, pretože táto funkcia v našej implementácii nemá využitie. Kľúč pre vyhľadávanie v tabuľke symbolov je zložený kľúč, kde najvyššiu prioritu má meno symbolu, potom počet argumentov a nakoniec typ identifikátora.

Užívateľské funkcie sa ukladajú do samostatnej tabuľky `g_func_symtable`, ktorá sa naplňa počas syntaktickej analýzy. Globálne premenné majú taktiež vlastnú tabuľku, ktorá sa však naplňa v sémantickej analýze. Pre prácu s lokálnymi premennými sme zvolili prístup, pri ktorom má každý blok kódu svoju vlastnú tabuľku symbolov. Tieto tabuľky symbolov sú vkladané do štruktúry `scope_stack`, ktorej položkami sú ukazovatele na korene tabuliek symbolov s lokálnymi premennými. Pre overenie existencie lokálnych premenných sa nad štruktúrou `scope_stack` volá funkcia `scope_stack_var_lookup`, ktorá od vrchu zásobníka až po jeho spodok prehľadá všetky tabuľky symbolov aktuálne prítomné na zásobníku a tým overí existenciu danej lokálnej premennej.

Celá tabuľka symbolov je implementovaná v súboroch `symtable.c/.h` a `scope stack` v súboroch `scope_stack.c/.h`

Generovanie kódu

Návrh a Implementácia

Generovanie kódu prebieha počas druhého prechodu AST. Na základe typu aktuálneho uzla sa volajú funkcie generujúce inštrukcie implementujúce jednotlivé jazykové konštrukcie, napr. `if`, `while` a ďalšie. Jedným z hlavných problémov bolo generovanie unikátnych návěstí a názvov premenných. Preto sa počas prvého prechodu AST v sémantickej analýze do uzlov zapisuje hodnota `code_gen_name`, ktorá predstavuje unikátny názov premennej v rámci danej funkcie. Táto vlastnosť je postačujúca, pretože každá funkcia má svoj vlastný lokálny frame.

Na generovanie unikátnych návěstí a dočasných premenných využívame globálnu inštanciu štruktúry `name_generator_t`, ktorá nesie informácie o polohe v AST, príznaky a hodnoty počítadiel. Pri volaní funkcií používame konvenciu Pascal. Pri implementácii sme prevažne používali dátový zásobník.

Generovanie kódu je implementované v dvoch súboroch: `code_gen.c/.h` a `built_in_funs.c/.h`. Súbor `built_in_funs.c` implementuje všetky vstavané funkcie, ktoré sa vždy vygenerujú do výstupného IFJcode25.

Odôvodnenie rozdelenia bodov

Rozdelenie bodov reflektuje skutočný čas a úsilie, ktoré jednotliví členovia tímu venovali projektu.

Kristián Cilling (22%) sa pripojil k projektu neskôr a venoval sa najmä PSA, dokumentácii, menšej časti generovania kódu a testovaniu svojich modulov.

Ostatní traja členovia (Martin Mezei, Martin Raček, Samuel Fačka) pracovali na projekte od začiatku a podieľali sa na ťažších častiach projektu (po 26%).

Chyby

1. Pri generovaní kódu nefunguje vnorené vetvetnie a vnorené cykly
2. Sémantická analýza nerozpozná samostatne stojacú globálnu premennú v riadku